

[DOC] pruebas

Documentación Técnica · AutoDocs AI · 12/06/2026

49	100	23	38
Archivos	Funciones	Clases	Endpoints

Score de Calidad: 66/100

Documentación del Proyecto

Generado automáticamente por **AutoDocs AI** — 2026-06-12 02:37 UTC

Resumen Ejecutivo

Este proyecto está desarrollado principalmente en **Python** y cuenta con 49 archivos fuente, 100 funciones, 23 clases y 38 endpoints de API documentados. El score de calidad del código es **66/100**.

Stack Tecnológico

Lenguaje	Cantidad
Python	42 archivos
HTML	1 archivos
CSS	2 archivos
JavaScript	4 archivos

Estadísticas del Proyecto

Métrica	Valor
Total de archivos	49
Funciones	100
Clases	23
Endpoints API	38

Score de calidad	66/100
Funciones documentadas	50 / 100
Clases documentadas	4 / 23

2. Documentación de Arquitectura y Diseño

2.1 Estructura de Directorios

```
proyecto/  
■■■ meditrack_modified/
```

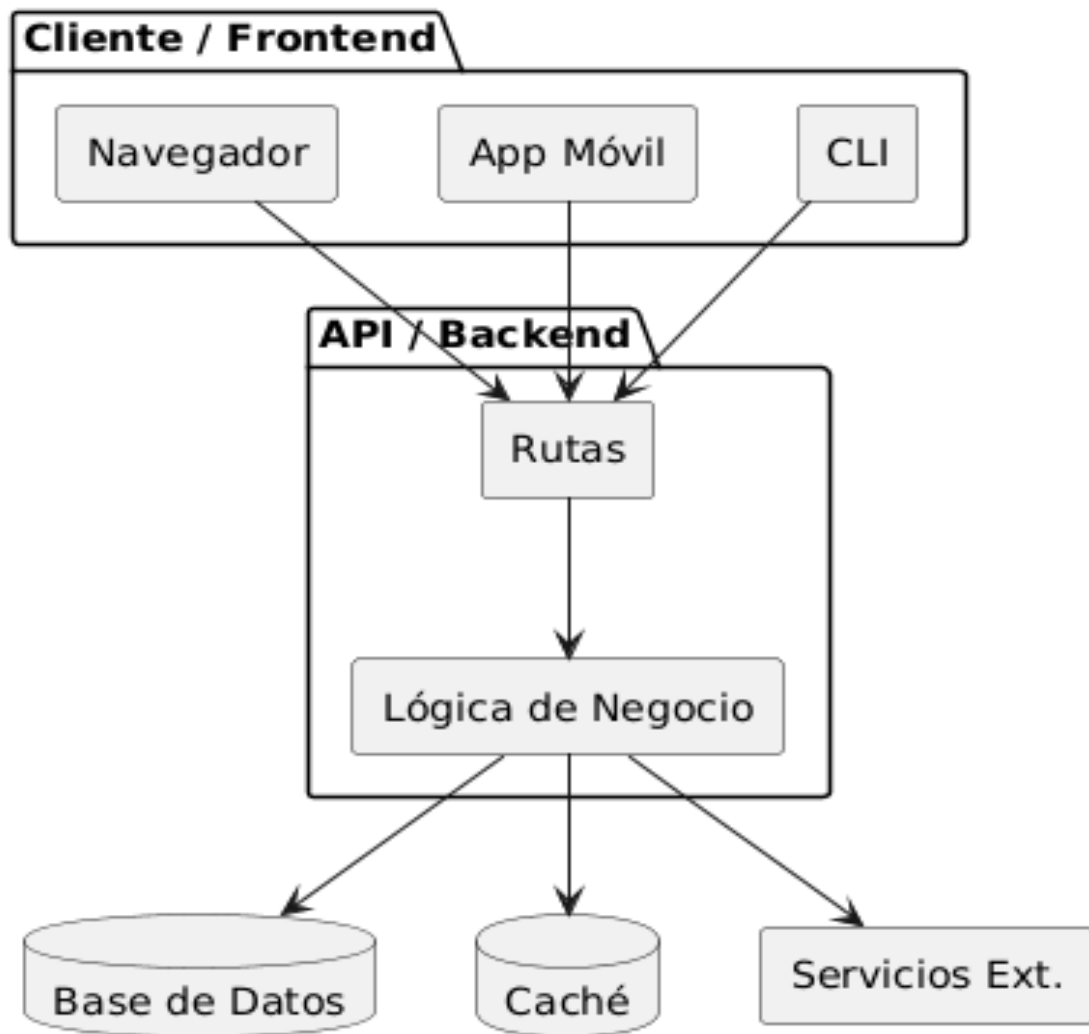
2.2 Distribución por Capas

- `meditrack_modified/` — 50 archivos [`.example(1)`, `.py(40)`, `.md(4)`, `.txt(1)`, `.html(1)`, `.css(2)`, `.js(1)`]

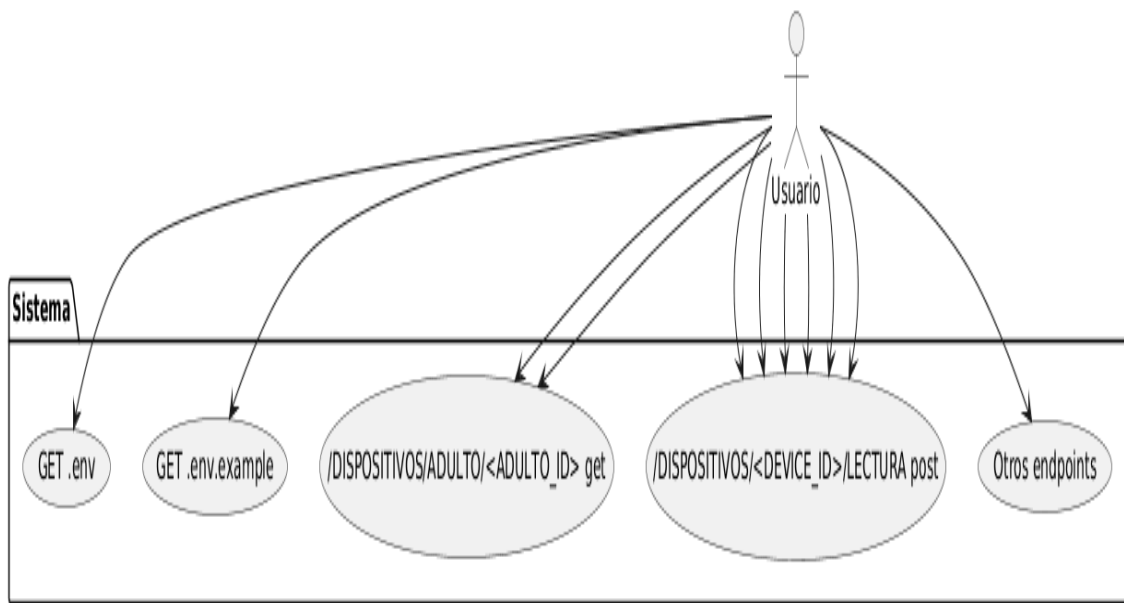
2.3 Distribución de Lenguajes

- Python: 42 archivos (86%)
- JavaScript: 4 archivos (8%)
- CSS: 2 archivos (4%)
- HTML: 1 archivos (2%)

2.4 Diagrama de Infraestructura



2.5 Diagrama de Casos de Uso



2.6 Diagrama de Clases



■ **Clases con información incompleta:** Las siguientes clases aparecen sin métodos ni atributos documentados: `BaseConfig`, `DevelopmentConfig`, `TestingConfig`, `ProductionConfig`. Para que el diagrama se visualice correctamente, agrega métodos, propiedades o docstrings a estas clases en tu código fuente y regenera la documentación.

2.7 Patrones Detectados

- **API REST:** 38 endpoints detectados (Flask/FastAPI, Django)
- **Patrón MVC:** clases Model detectadas
- **Programación Orientada a Objetos:** constructores detectados

2.8 Insights de Arquitectura (IA)

Análisis del Proyecto

- **Patrones arquitectónicos:** Probablemente se utilice un patrón monolítico debido a la cantidad de funciones y clases en un solo lenguaje (Python).
 - **Buenas prácticas:** Se detectan HTML, CSS y JavaScript, lo que sugiere una capa de presentación adecuada.
 - **Áreas de mejora:** La cantidad de endpoints (38) sugiere una posible sobrecarga en la capa de servicios. Se recomienda revisar y optimizar.
 - **Complejidad:** La complejidad es moderada debido a la cantidad de funciones y clases.
 - **Recomendaciones de escalabilidad:** Se sugiere considerar un patrón de microservicios para distribuir la carga y mejorar la escalabilidad. Además, se recomienda implementar principios de diseño de software como SOLID para mejorar la mantenibilidad y flexibilidad del código.
-

3. Documentación Técnica de Código

3.1 Guía de Configuración del Entorno de Desarrollo

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Crear entorno virtual (recomendado para Python)
python -m venv venv
source venv/bin/activate # Linux/Mac
venv\Scripts\activate   # Windows

# 3. Instalar dependencias
pip install -r requirements.txt

# 4. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores locales

# 5. Ejecutar en modo desarrollo
python app.py # o npm run dev
```

3.2 Documentación de API

Total de endpoints detectados: **38**

Módulo: `meditrack_modified\db_setup.py`

[GET] `GET .env`

- **Framework:** Django
- **Archivo:** `meditrack_modified\db_setup.py`

Descripción del Endpoint de API

Propósito del Endpoint

El propósito de este endpoint es obtener información importante del proyecto de manera segura. Imagina que tienes un cajón secreto donde guardas tus cosas más valiosas, y este endpoint es como una llave que te permite acceder a ese cajón de manera controlada.

Funcionalidad

Este endpoint se utiliza para recuperar configuraciones y variables de entorno del proyecto. En otras palabras, es como preguntar a alguien "¿Cuáles son las reglas y ajustes que debo seguir para hacer que este proyecto funcione correctamente?". La respuesta que obtienes es un conjunto de instrucciones y ajustes que el proyecto necesita para funcionar de manera adecuada.

Información Necesaria

Para que este endpoint funcione, no se necesita proporcionar mucha información. Básicamente, solo necesitas saber la ruta correcta para acceder a él (en este caso, `.env`). Es como saber la dirección de una casa para poder visitarla.

Resultado

El resultado que devuelves al utilizar este endpoint es un conjunto de configuraciones y variables de entorno que el proyecto utiliza. Es como recibir un paquete con instrucciones detalladas sobre cómo configurar y ajustar el proyecto para que funcione correctamente.

Uso en la Vida Real

En la vida real, este tipo de endpoint se utiliza

Parámetros esperados:

- `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

- `200` — Operación exitosa
- `400` — Solicitud incorrecta
- `401` — No autorizado
- `404` — Recurso no encontrado
- `500` — Error interno del servidor

[GET] GET .env.example

- **Framework:** Django
- **Archivo:** meditrack_modified\db_setup.py

Descripción del Endpoint

Propósito del endpoint

El propósito de este endpoint es proporcionar un ejemplo de cómo configurar un entorno de desarrollo para un proyecto de software. En otras palabras, es como una guía para ayudar a los desarrolladores a configurar su entorno de trabajo de manera correcta.

Funcionalidad

Este endpoint se utiliza para obtener un archivo de ejemplo que contiene configuraciones básicas para un proyecto de software. El archivo se llama ".env.example" y contiene información necesaria para que el proyecto funcione correctamente.

¿Qué hace este endpoint?

Este endpoint simplemente devuelve un archivo de ejemplo que contiene configuraciones básicas para un proyecto de software. No realiza cálculos complejos ni procesa grandes cantidades de datos. Simplemente proporciona un archivo que puede ser utilizado como referencia para configurar el entorno de desarrollo.

Información necesaria

Para que este endpoint funcione, no se necesita proporcionar ninguna información adicional. Simplemente se necesita realizar una solicitud GET al endpoint y el archivo de ejemplo se devolverá automáticamente.

Resultado

El resultado de este endpoint es un archivo de texto que contiene configuraciones básicas para un proyecto de software. El archivo puede ser utilizado como referencia para configurar el entorno de desarrollo y asegurarse de que el proyecto funcione correctamente.

Uso en la vida real

En la vida real, este endpoint se utiliza para ayudar

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- **200** — Operación exitosa

- 400 — Solicitud incorrecta
 - 401 — No autorizado
 - 404 — Recurso no encontrado
 - 500 — Error interno del servidor
-

Módulo: `meditrack_modified\app\__init__.py`

[GET] `/API/V1/HEALTH` `get`

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\__init__.py`

Endpoint de Salud de la API

1. Propósito del endpoint:

El propósito de este endpoint es verificar si la aplicación está funcionando correctamente. Es como hacer una prueba de sonido en un sistema de audio para asegurarse de que todo esté en orden.

2. Funcionalidad:

Este endpoint realiza una comprobación rápida para determinar si la aplicación está en buen estado y lista para funcionar. Es como un chequeo de rutina que verifica que todos los componentes estén funcionando como se espera.

3. Información necesaria:

No se requiere ninguna información especial para que este endpoint funcione. Simplemente se envía una solicitud a la aplicación para que responda con un resultado.

4. Resultado:

El resultado que devuelve este endpoint es una respuesta simple que indica si la aplicación está funcionando correctamente o no. Es como un mensaje que dice "todo está bien" o "hay un problema".

5. Uso en la vida real:

En la vida real, este tipo de endpoint se utiliza para:

- **Monitorear el estado de la aplicación:** Los administradores de sistemas pueden utilizar este endpoint para verificar si la aplicación está funcionando correctamente y detectar cualquier problema potencial antes de que afecte a los usuarios.
- **Realizar pruebas de rendimiento:** Los desarrolladores pueden utilizar este endpoint para probar la velocidad y la estabilidad de

Parámetros esperados:

- `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
- 400 — Solicitud incorrecta

- [401](#) — No autorizado
 - [404](#) — Recurso no encontrado
 - [500](#) — Error interno del servidor
-

Módulo: `meditrack_modified\app\controllers\auth_controller.py`

[GET] `/REGISTRO post`

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\controllers\auth_controller.py`

Endpoint `/REGISTRO post`

Parámetros esperados:

- [Authorization](#) (Header, requerido): Bearer JWT token

Códigos de respuesta:

- [200](#) — Operación exitosa
 - [400](#) — Solicitud incorrecta
 - [401](#) — No autorizado
 - [404](#) — Recurso no encontrado
 - [500](#) — Error interno del servidor
-

[GET] `/LOGIN post`

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\controllers\auth_controller.py`

Endpoint `/LOGIN post`

Parámetros esperados:

- [Authorization](#) (Header, requerido): Bearer JWT token

Códigos de respuesta:

- [200](#) — Operación exitosa
 - [400](#) — Solicitud incorrecta
 - [401](#) — No autorizado
 - [404](#) — Recurso no encontrado
 - [500](#) — Error interno del servidor
-

[GET] `/REFRESH post`

- **Framework:** Flask/FastAPI

- **Archivo:** `meditrack_modified\app\controllers\auth_controller.py`

Endpoint `/REFRESH` post

Parámetros esperados:

- `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

- `200` — Operación exitosa
 - `400` — Solicitud incorrecta
 - `401` — No autorizado
 - `404` — Recurso no encontrado
 - `500` — Error interno del servidor
-

[GET] `/CAMBIAR-PASSWORD` post

- **Framework:** Flask/FastAPI

- **Archivo:** `meditrack_modified\app\controllers\auth_controller.py`

Endpoint `/CAMBIAR-PASSWORD` post

Parámetros esperados:

- `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

- `200` — Operación exitosa
 - `400` — Solicitud incorrecta
 - `401` — No autorizado
 - `404` — Recurso no encontrado
 - `500` — Error interno del servidor
-

Módulo: `meditrack_modified\app\controllers\iot_controller.py`

[GET] `/DISPOSITIVOS` post

- **Framework:** Flask/FastAPI

- **Archivo:** `meditrack_modified\app\controllers\iot_controller.py`

Endpoint `/DISPOSITIVOS` post

Parámetros esperados:

- `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
 - 400 — Solicitud incorrecta
 - 401 — No autorizado
 - 404 — Recurso no encontrado
 - 500 — Error interno del servidor
-

[GET] /DISPOSITIVOS/ADULTO/<ADULTO_ID> get

- **Framework:** Flask/FastAPI
- **Archivo:** meditrack_modified\app\controllers\iot_controller.py

Endpoint /DISPOSITIVOS/ADULTO/<ADULTO_ID> get

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
 - 400 — Solicitud incorrecta
 - 401 — No autorizado
 - 404 — Recurso no encontrado
 - 500 — Error interno del servidor
-

[GET] /DISPOSITIVOS/<DEVICE_ID>/LECTURA post

- **Framework:** Flask/FastAPI
- **Archivo:** meditrack_modified\app\controllers\iot_controller.py

Endpoint /DISPOSITIVOS/<DEVICE_ID>/LECTURA post

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
 - 400 — Solicitud incorrecta
 - 401 — No autorizado
 - 404 — Recurso no encontrado
 - 500 — Error interno del servidor
-

[GET] /DISPOSITIVOS/<DEVICE_ID>/LECTURAS get

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\controllers\iot_controller.py`

Endpoint `/DISPOSITIVOS/<DEVICE_ID>/LECTURAS` get

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- **200** — Operación exitosa
- **400** — Solicitud incorrecta
- **401** — No autorizado
- **404** — Recurso no encontrado
- **500** — Error interno del servidor

[GET] `/DISPOSITIVOS/<DEVICE_ID>/ULTIMA-LECTURA` get

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\controllers\iot_controller.py`

Endpoint `/DISPOSITIVOS/<DEVICE_ID>/ULTIMA-LECTURA` get

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- **200** — Operación exitosa
- **400** — Solicitud incorrecta
- **401** — No autorizado
- **404** — Recurso no encontrado
- **500** — Error interno del servidor

[GET] `/ALERTAS/ADULTO/<ADULTO_ID>` get

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\controllers\iot_controller.py`

Endpoint `/ALERTAS/ADULTO/<ADULTO_ID>` get

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
 - 400 — Solicitud incorrecta
 - 401 — No autorizado
 - 404 — Recurso no encontrado
 - 500 — Error interno del servidor
-

[GET] /ESTADO get

- **Framework:** Flask/FastAPI
- **Archivo:** meditrack_modified\app\controllers\iot_controller.py

Endpoint /ESTADO get

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
 - 400 — Solicitud incorrecta
 - 401 — No autorizado
 - 404 — Recurso no encontrado
 - 500 — Error interno del servidor
-

Módulo: meditrack_modified\app\controllers\medication_controller.py

[GET] / post

- **Framework:** Flask/FastAPI
- **Archivo:** meditrack_modified\app\controllers\medication_controller.py

Endpoint / post

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
 - 400 — Solicitud incorrecta
 - 401 — No autorizado
 - 404 — Recurso no encontrado
 - 500 — Error interno del servidor
-

[GET] /ADULTO/<ADULTO_ID> get

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\controllers\medication_controller.py`

Parámetros esperados:

- `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
 - 400 — Solicitud incorrecta
 - 401 — No autorizado
 - 404 — Recurso no encontrado
 - 500 — Error interno del servidor
-

[GET] /<MED_ID> get

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\controllers\medication_controller.py`

Parámetros esperados:

- `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
 - 400 — Solicitud incorrecta
 - 401 — No autorizado
 - 404 — Recurso no encontrado
 - 500 — Error interno del servidor
-

[GET] /<MED_ID> put

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\controllers\medication_controller.py`

Parámetros esperados:

- `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
- 400 — Solicitud incorrecta
- 401 — No autorizado

- [404](#) — Recurso no encontrado
 - [500](#) — Error interno del servidor
-

[GET] /<MED_ID> delete

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\controllers\medication_controller.py`

Parámetros esperados:

- [Authorization](#) (Header, requerido): Bearer JWT token

Códigos de respuesta:

- [200](#) — Operación exitosa
 - [400](#) — Solicitud incorrecta
 - [401](#) — No autorizado
 - [404](#) — Recurso no encontrado
 - [500](#) — Error interno del servidor
-

Módulo: `meditrack_modified\app\controllers\medication_log_controller.py`

[GET] / post

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\controllers\medication_log_controller.py`

Parámetros esperados:

- [Authorization](#) (Header, requerido): Bearer JWT token

Códigos de respuesta:

- [200](#) — Operación exitosa
 - [400](#) — Solicitud incorrecta
 - [401](#) — No autorizado
 - [404](#) — Recurso no encontrado
 - [500](#) — Error interno del servidor
-

[GET] /ADULTO/<ADULTO_ID> get

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\controllers\medication_log_controller.py`

Parámetros esperados:

- [Authorization](#) (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
 - 400 — Solicitud incorrecta
 - 401 — No autorizado
 - 404 — Recurso no encontrado
 - 500 — Error interno del servidor
-

[GET] /ADULTO/<ADULTO_ID>/HOY get

- **Framework:** Flask/FastAPI
- **Archivo:** meditrack_modified\app\controllers\medication_log_controller.py

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
 - 400 — Solicitud incorrecta
 - 401 — No autorizado
 - 404 — Recurso no encontrado
 - 500 — Error interno del servidor
-

[GET] /ADULTO/<ADULTO_ID>/ADHERENCIA get

- **Framework:** Flask/FastAPI
- **Archivo:** meditrack_modified\app\controllers\medication_log_controller.py

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
 - 400 — Solicitud incorrecta
 - 401 — No autorizado
 - 404 — Recurso no encontrado
 - 500 — Error interno del servidor
-

Módulo: meditrack_modified\app\controllers\notification_controller.py

[GET] / get

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\controllers\notification_controller.py`

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- **200** — Operación exitosa
 - **400** — Solicitud incorrecta
 - **401** — No autorizado
 - **404** — Recurso no encontrado
 - **500** — Error interno del servidor
-

[GET] `/CONTEO` get

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\controllers\notification_controller.py`

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- **200** — Operación exitosa
 - **400** — Solicitud incorrecta
 - **401** — No autorizado
 - **404** — Recurso no encontrado
 - **500** — Error interno del servidor
-

[GET] `/<NOTIF_ID>/LEIDA` put

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\controllers\notification_controller.py`

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- **200** — Operación exitosa
- **400** — Solicitud incorrecta
- **401** — No autorizado
- **404** — Recurso no encontrado

- 500 — Error interno del servidor

[GET] /TODAS-LEIDAS put

- **Framework:** Flask/FastAPI
- **Archivo:** meditrack_modified\app\controllers\notification_controller.py

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
- 400 — Solicitud incorrecta
- 401 — No autorizado
- 404 — Recurso no encontrado
- 500 — Error interno del servidor

Módulo: meditrack_modified\app\controllers\user_controller.py

[GET] / get

- **Framework:** Flask/FastAPI
- **Archivo:** meditrack_modified\app\controllers\user_controller.py

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
- 400 — Solicitud incorrecta
- 401 — No autorizado
- 404 — Recurso no encontrado
- 500 — Error interno del servidor

[GET] /ME get

- **Framework:** Flask/FastAPI
- **Archivo:** meditrack_modified\app\controllers\user_controller.py

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- [200](#) — Operación exitosa
 - [400](#) — Solicitud incorrecta
 - [401](#) — No autorizado
 - [404](#) — Recurso no encontrado
 - [500](#) — Error interno del servidor
-

[GET] /ME put

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\controllers\user_controller.py`

Parámetros esperados:

- [Authorization](#) (Header, requerido): Bearer JWT token

Códigos de respuesta:

- [200](#) — Operación exitosa
 - [400](#) — Solicitud incorrecta
 - [401](#) — No autorizado
 - [404](#) — Recurso no encontrado
 - [500](#) — Error interno del servidor
-

[GET] /<USER_ID> get

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\controllers\user_controller.py`

Parámetros esperados:

- [Authorization](#) (Header, requerido): Bearer JWT token

Códigos de respuesta:

- [200](#) — Operación exitosa
 - [400](#) — Solicitud incorrecta
 - [401](#) — No autorizado
 - [404](#) — Recurso no encontrado
 - [500](#) — Error interno del servidor
-

[GET] /<USER_ID> delete

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\controllers\user_controller.py`

Parámetros esperados:

- [Authorization](#) (Header, requerido): Bearer JWT token

Códigos de respuesta:

- [200](#) — Operación exitosa
 - [400](#) — Solicitud incorrecta
 - [401](#) — No autorizado
 - [404](#) — Recurso no encontrado
 - [500](#) — Error interno del servidor
-

[GET] /<USER_ID>/ACTIVAR post

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\controllers\user_controller.py`

Parámetros esperados:

- [Authorization](#) (Header, requerido): Bearer JWT token

Códigos de respuesta:

- [200](#) — Operación exitosa
 - [400](#) — Solicitud incorrecta
 - [401](#) — No autorizado
 - [404](#) — Recurso no encontrado
 - [500](#) — Error interno del servidor
-

[GET] /<ADULTO_ID>/VINCULAR-FAMILIAR/<FAMILIAR_ID> post

- **Framework:** Flask/FastAPI
- **Archivo:** `meditrack_modified\app\controllers\user_controller.py`

Parámetros esperados:

- [Authorization](#) (Header, requerido): Bearer JWT token

Códigos de respuesta:

- [200](#) — Operación exitosa
 - [400](#) — Solicitud incorrecta
 - [401](#) — No autorizado
 - [404](#) — Recurso no encontrado
 - [500](#) — Error interno del servidor
-

[GET] /<ADULTO_ID>/VINCULAR-FAMILIAR/<FAMILIAR_ID> delete

- **Framework:** Flask/FastAPI
- **Archivo:** meditrack_modified\app\controllers\user_controller.py

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
 - 400 — Solicitud incorrecta
 - 401 — No autorizado
 - 404 — Recurso no encontrado
 - 500 — Error interno del servidor
-

[GET] /<ADULTO_ID>/ASIGNAR-MEDICO/<MEDICO_ID> post

- **Framework:** Flask/FastAPI
- **Archivo:** meditrack_modified\app\controllers\user_controller.py

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
 - 400 — Solicitud incorrecta
 - 401 — No autorizado
 - 404 — Recurso no encontrado
 - 500 — Error interno del servidor
-

[GET] /<ADULTO_ID>/FAMILIARES get

- **Framework:** Flask/FastAPI
- **Archivo:** meditrack_modified\app\controllers\user_controller.py

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
- 400 — Solicitud incorrecta
- 401 — No autorizado

- 404 — Recurso no encontrado
- 500 — Error interno del servidor

[GET] /ME/PACIENTES get

- **Framework:** Flask/FastAPI
- **Archivo:** meditrack_modified\app\controllers\user_controller.py

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
- 400 — Solicitud incorrecta
- 401 — No autorizado
- 404 — Recurso no encontrado
- 500 — Error interno del servidor

3.3 Estado de Comentarios y Docstrings

Total de funciones: 100

- [OK] Documentadas: 50 (50%)
- [X] Sin documentar: 50 (50%)

Visualización de documentación:

[██]
50% documentado

Ejemplo de docstrings detectados:

```
def print_header(text):  
    """  
    Imprime un encabezado con estilo.  
    """  
    ...
```

```
def print_success(text):  
    """  
    Imprime mensaje de éxito.  
    """  
    ...
```

```
def print_error(text):  
    """  
    Imprime mensaje de error.
```

```
"""  
...
```

Funciones que requieren documentación:

- `seed()` en `meditrack_modified\seed.py`
- `health()` en `meditrack_modified\app\__init__.py`
- `not_found(e)` en `meditrack_modified\app\__init__.py`
- `method_not_allowed(e)` en `meditrack_modified\app\__init__.py`
- `internal_error(e)` en `meditrack_modified\app\__init__.py`
- ... y 45 más

3.4 Planes de Prueba (Testing)

No se detectaron pruebas automatizadas. Se recomienda implementar:

```
# Ejemplo de estructura de tests recomendada  
import pytest  
  
class TestEndpoints:  
    def test_get_returns_200(self, client):  
        response = client.get('/api/endpoint')  
        assert response.status_code == 200  
  
    def test_post_creates_resource(self, client):  
        data = {'name': 'test'}  
        response = client.post('/api/endpoint', json=data)  
        assert response.status_code == 201  
  
    def test_unauthorized_returns_401(self, client):  
        response = client.get('/api/protected')  
        assert response.status_code == 401
```

Escenarios de prueba recomendados:

Escenario	Tipo	Prioridad
GET .env — respuesta 200	Integración	Alta
GET .env — sin auth (401)	Seguridad	Alta
GET .env.example — respuesta 200	Integración	Alta
GET .env.example — sin auth (401)	Seguridad	Alta
/API/V1/HEALTH get — respuesta 200	Integración	Alta
/API/V1/HEALTH get — sin auth (401)	Seguridad	Alta
/REGISTRO post — respuesta 200	Integración	Alta
/REGISTRO post — sin auth (401)	Seguridad	Alta
/LOGIN post — respuesta 200	Integración	Alta

/LOGIN post — sin auth (401)	Seguridad	Alta
/REFRESH post — respuesta 200	Integración	Alta
/REFRESH post — sin auth (401)	Seguridad	Alta

4. Documentación de Clases

Total: **23 clases**

BaseConfig

- **Archivo:** `meditrack_modified\app\config\settings.py`
- **Línea:** 12
- **Hereda de:** object
- **Métodos:** 0
- **Documentada:** [OK] Sí

Introducción a la Clase BaseConfig

La clase BaseConfig es una parte fundamental de un proyecto en Python que se utiliza para configurar y personalizar el comportamiento de la aplicación. En este sentido, podemos considerarla como un "manual de instrucciones" que establece las reglas y parámetros básicos para que el sistema funcione correctamente.

Propósito de la clase

Imagine que estás construyendo una casa y necesitas una guía que te diga cómo deben ser las paredes, el techo y las puertas. De la misma manera, la clase BaseConfig es como una guía que establece las bases para que el sistema funcione de manera eficiente y segura. Su propósito es proporcionar una configuración básica que se puede compartir en todos los entornos y situaciones.

Qué representa esta clase en el sistema

La clase BaseConfig se puede comparar con un "conjunto de reglas" que se aplican a todo el sistema. Al igual que un conjunto de reglas de tráfico que se deben seguir en una ciudad, la BaseConfig establece las normas y parámetros que deben ser respetados por todas las partes del sistema. Esto garantiza que todo funcione de manera coherente y sin conflictos.

Qué puede hacer esta clase

La clase BaseConfig puede realizar varias tareas importantes, como:

- Establecer parámetros bás

DevelopmentConfig

- **Archivo:** `meditrack_modified\app\config\settings.py`

- **Línea:** 69
- **Hereda de:** BaseConfig
- **Métodos:** 0
- **Documentada:** [X] No

Descripción de la clase DevelopmentConfig

1. Propósito de la clase:

La clase DevelopmentConfig es como un conjunto de instrucciones que ayuda a configurar y personalizar un proyecto de software. Piensa en ella como un manual de configuración que dice al proyecto cómo debe funcionar y qué características debe tener.

2. Representación en el sistema:

Imagina que estás construyendo una casa. La clase DevelopmentConfig es como un plano de la casa que muestra cómo deben estar dispuestas las habitaciones, las ventanas y las puertas. En este caso, el plano (la clase) muestra cómo debe estar configurado el proyecto de software para que funcione correctamente.

3. Funcionalidades principales:

La clase DevelopmentConfig puede hacer varias cosas importantes, como:

- * Establecer los parámetros básicos del proyecto, como la forma en que se almacenan y recuperan los datos.
- * Configurar la seguridad del proyecto, como la forma en que se autenticarán los usuarios.
- * Definir cómo se deben manejar los errores y las excepciones en el proyecto.

En resumen, esta clase ayuda a configurar el proyecto para que funcione de manera eficiente y segura.

4. Ejemplos prácticos:

En la vida real, la clase DevelopmentConfig se utiliza en proyectos de software como:

- * Un sitio web que necesita configurar la seguridad para proteger la

TestingConfig

- **Archivo:** meditrack_modified\app\config\settings.py
- **Línea:** 75
- **Hereda de:** BaseConfig
- **Métodos:** 0
- **Documentada:** [X] No

Introducción a la clase TestingConfig

La clase TestingConfig es una parte importante de un proyecto de software llamado Meditrack. Aunque su nombre puede parecer complicado, su propósito es bastante sencillo.

1. Propósito de la clase:

La clase `TestingConfig` se utiliza para configurar y personalizar la forma en que se prueban y verifican las funcionalidades de un sistema. Piensa en ello como un conjunto de instrucciones que dicen: "¿Cómo queremos probar y verificar que todo funcione correctamente?"

2. Representación en el sistema:

Imagina que estás construyendo un coche. La clase `TestingConfig` sería como el manual de instrucciones que te dice cómo probar y verificar que todos los componentes del coche funcionen correctamente antes de salir a la carretera. En este caso, la clase `TestingConfig` es como el manual de instrucciones para el sistema `Meditrack`.

3. Funcionalidades principales:

La clase `TestingConfig` puede hacer varias cosas, como:

- Establecer parámetros y configuraciones para las pruebas
- Definir cómo se deben realizar las pruebas
- Proporcionar información sobre qué se debe probar y cómo se debe hacer

Piensa en ello como un conjunto de preguntas que se responden automáticamente cuando se prueban las funcionalidades del sistema: "¿

`ProductionConfig`

- **Archivo:** `meditrack_modified\app\config\settings.py`
- **Línea:** 83
- **Hereda de:** `BaseConfig`
- **Métodos:** 0
- **Documentada:** [X] No

Clase `ProductionConfig`

`IoTDeviceModel`

- **Archivo:** `meditrack_modified\app\models\iot_device.py`
- **Línea:** 21
- **Hereda de:** `object`
- **Métodos:** 2
- **Documentada:** [X] No

Clase `IoTDeviceModel`

Métodos:

- `schema()`
 - `to_response()`
-

IoTReadingModel

- **Archivo:** meditrack_modified\app\models\iot_device.py
- **Línea:** 71
- **Hereda de:** object
- **Métodos:** 2
- **Documentada:** [OK] Sí

Lectura individual de un dispositivo IoT.

Métodos:

- `schema()`
 - `to_response()`
-

MedicationModel

- **Archivo:** meditrack_modified\app\models\medication.py
- **Línea:** 9
- **Hereda de:** object
- **Métodos:** 2
- **Documentada:** [X] No

Clase MedicationModel

Métodos:

- `schema()`
 - `to_response()`
-

MedicationLogModel

- **Archivo:** meditrack_modified\app\models\medication_log.py
- **Línea:** 9
- **Hereda de:** object
- **Métodos:** 2
- **Documentada:** [X] No

Clase MedicationLogModel

Métodos:

- `schema()`
- `to_response()`

NotificationModel

- **Archivo:** meditrack_modified\app\models\notification.py
- **Línea:** 9
- **Hereda de:** object
- **Métodos:** 2
- **Documentada:** [X] No

Clase NotificationModel

Métodos:

- `schema()`
 - `to_response()`
-

UserModel

- **Archivo:** meditrack_modified\app\models\user.py
- **Línea:** 9
- **Hereda de:** object
- **Métodos:** 2
- **Documentada:** [OK] Sí

Esquema lógico del documento Usuario en MongoDB.

Métodos:

- `schema()`
 - `to_response()`
-

BaseRepository

- **Archivo:** meditrack_modified\app\repository\base_repository.py
- **Línea:** 19
- **Hereda de:** object
- **Métodos:** 11
- **Documentada:** [OK] Sí

CRUD genérico para una colección de MongoDB.

Métodos:

- `__init__()`

- `create()`
 - `find_by_id()`
 - `find_one()`
 - `find_many()`
 - `count()`
 - `update_by_id()`
 - `update_one()`
 - `soft_delete()`
 - `hard_delete()`
 - `exists()`
-

IoTDeviceRepository

- **Archivo:** `meditrack_modified\app\repository\iot_repository.py`
- **Línea:** 11
- **Hereda de:** BaseRepository
- **Métodos:** 1
- **Documentada:** [X] No

Clase IoTDeviceRepository

Métodos:

- `find_by_adulto()`
-

IoTReadingRepository

- **Archivo:** `meditrack_modified\app\repository\iot_repository.py`
- **Línea:** 19
- **Hereda de:** BaseRepository
- **Métodos:** 3
- **Documentada:** [X] No

Clase IoTReadingRepository

Métodos:

- `find_by_device()`
 - `last_reading()`
 - `alertas_recientes()`
-

MedicationLogRepository

- **Archivo:** meditrack_modified\app\repository\medication_log_repository.py
- **Línea:** 10
- **Hereda de:** BaseRepository
- **Métodos:** 5
- **Documentada:** [X] No

Clase MedicationLogRepository

Métodos:

- find_by_adulto()
 - find_by_medicamento()
 - tomas_hoy()
 - adherencia()
 - omisiones_recientes()
-

MedicationRepository

- **Archivo:** meditrack_modified\app\repository\medication_repository.py
- **Línea:** 9
- **Hereda de:** BaseRepository
- **Métodos:** 2
- **Documentada:** [X] No

Clase MedicationRepository

Métodos:

- find_by_adulto()
 - find_activos_con_horario()
-

NotificationRepository

- **Archivo:** meditrack_modified\app\repository\notification_repository.py
- **Línea:** 9
- **Hereda de:** BaseRepository
- **Métodos:** 4
- **Documentada:** [X] No

Métodos:

- find_by_usuario()

- `marcar_leida()`
 - `marcar_todas_leidas()`
 - `no_leidas_count()`
-

UserRepository

- **Archivo:** `meditrack_modified\app\repository\user_repository.py`
- **Línea:** 9
- **Hereda de:** BaseRepository
- **Métodos:** 7
- **Documentada:** [X] No

Métodos:

- `find_by_email()`
 - `find_by_rol()`
 - `link_familiar()`
 - `unlink_familiar()`
 - `assign_medico()`
 - `get_familiares_of_adulto()`
 - `update_last_login()`
-

AuthService

- **Archivo:** `meditrack_modified\app\services\auth_service.py`
- **Línea:** 18
- **Hereda de:** object
- **Métodos:** 5
- **Documentada:** [X] No

Métodos:

- `__init__()`
 - `register()`
 - `login()`
 - `refresh_token()`
 - `change_password()`
-

IoTService

- **Archivo:** meditrack_modified\app\services\iot_service.py
- **Línea:** 40
- **Hereda de:** object
- **Métodos:** 10
- **Documentada:** [X] No

Métodos:

- `__init__()`
 - `registrar_dispositivo()`
 - `mis_dispositivos()`
 - `actualizar_estado()`
 - `registrar_lectura()`
 - `historial_lecturas()`
 - `ultima_lectura()`
 - `alertas_recientes()`
 - `_detectar_alerta()`
 - `conectar_mqtt()`
-

MedicationLogService

- **Archivo:** meditrack_modified\app\services\medication_log_service.py
- **Línea:** 15
- **Hereda de:** object
- **Métodos:** 5
- **Documentada:** [X] No

Métodos:

- `__init__()`
 - `registrar_toma()`
 - `historial()`
 - `tomas_hoy()`
 - `adherencia()`
-

MedicationService

- **Archivo:** meditrack_modified\app\services\medication_service.py
- **Línea:** 14
- **Hereda de:** object

- **Métodos:** 6
- **Documentada:** [X] No

Métodos:

- `__init__()`
 - `create_medication()`
 - `get_medications()`
 - `get_medication()`
 - `update_medication()`
 - `deactivate_medication()`
-

NotificationService

- **Archivo:** `meditrack_modified\app\services\notification_service.py`
- **Línea:** 14
- **Hereda de:** `object`
- **Métodos:** 10
- **Documentada:** [X] No

Métodos:

- `__init__()`
 - `crear_notificacion()`
 - `notificar_omision()`
 - `notificar_recordatorio()`
 - `notificar_alerta_iot()`
 - `mis_notificaciones()`
 - `marcar_leida()`
 - `marcar_todas_leidas()`
 - `no_leidas_count()`
 - `_enviar_email()`
-

UserService

- **Archivo:** `meditrack_modified\app\services\user_service.py`
- **Línea:** 10
- **Hereda de:** `object`
- **Métodos:** 11
- **Documentada:** [X] No

Métodos:

- `__init__()`
 - `get_user()`
 - `list_users()`
 - `update_profile()`
 - `deactivate_user()`
 - `activate_user()`
 - `vincular_familiar()`
 - `desvincular_familiar()`
 - `asignar_medico()`
 - `get_familiares()`
 - `get_mis_pacientes()`
-
-

5. Documentación de Funciones

Total: **100 funciones**

Módulo: `meditrack_modified\db_setup.py`

`print_header`

- Parámetros: `text`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

La función `print_header` se utiliza para mostrar un título o encabezado en una pantalla. Para funcionar, necesita un texto que se desee mostrar. Una vez recibido el texto, imprime el encabezado de manera atractiva. En la vida real, se usa para presentar información de manera clara y organizada, como títulos de secciones en un informe o encabezados en un sitio web. En el sistema, es importante porque ayuda a mantener la información organizada y fácil de leer, lo que mejora la experiencia del usuario. Su función es similar a cuando se escribe un título en una página para que sea más fácil de entender y leer.

`print_success`

- Parámetros: `text`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

La función `print_success` muestra un mensaje cuando algo se ha realizado con éxito. Para funcionar, necesita un texto que indicará qué ha sucedido. Cuando se ejecuta, imprime este texto en la pantalla. En la vida real, se usa para informar al usuario que una acción ha sido completada, como cuando se crea una cuenta o se guarda un archivo. Es importante en el sistema porque ayuda a los usuarios a entender qué ha pasado y si deben tomar alguna otra acción. Es una forma de proporcionar retroalimentación clara y útil.

`print_error`

- Parámetros: `text`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

La función "print_error" sirve para mostrar mensajes de error de manera clara. Para funcionar, necesita un texto que describa el error. El resultado es un mensaje visible que indica qué salió mal. En la vida real, se usa para informar a los usuarios sobre problemas en el sistema, como errores de conexión o datos incorrectos. Esta función es importante porque ayuda a identificar y solucionar problemas de manera rápida y efectiva, mejorando la experiencia del usuario. Al mostrar mensajes de error claros, permite a los desarrolladores y usuarios comprender y corregir errores, lo que a su vez mejora la estabilidad y confiabilidad del sistema.

`print_info`

- Parámetros: `text`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

La función **print_info** muestra un mensaje en la pantalla.

Necesita un **texto** para funcionar.

Muestra el mensaje en la pantalla.

Se usa para informar al usuario sobre algo, como un estado o resultado.

Es importante porque ayuda a los usuarios a entender lo que está sucediendo en el sistema.

`print_warning`

- Parámetros: `text`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

La función "print_warning" es como un mensajero que avisa sobre algo importante. Su propósito es mostrar un mensaje de advertencia para que el usuario sepa que algo no está bien. Necesita un texto para saber qué mensaje mostrar. Cuando se utiliza, imprime el mensaje de advertencia en la pantalla. En la vida real, se usa para alertar sobre errores o problemas en un programa. Por ejemplo, si un usuario introduce una fecha incorrecta, la función puede mostrar un mensaje de advertencia para corregir el error. En el sistema, es importante porque ayuda a prevenir errores y a mantener la seguridad de los datos. De esta forma, los usuarios pueden tomar medidas para solucionar el problema y evitar daños.

check_mongodb_connection

- Parámetros: —
- Async: —
- Complejidad: [LOW] 4
- Documentada: [OK]

Verifica la conexión a MongoDB.

create_collections_and_indexes

- Parámetros: —
- Async: —
- Complejidad: [MED] 10
- Documentada: [OK]

Crea colecciones e índices.

check_environment

- Parámetros: —
- Async: —
- Complejidad: [LOW] 3
- Documentada: [OK]

La función "check_environment" verifica que todo esté listo para que el programa funcione correctamente.

Necesita saber cómo está configurado el entorno para hacer esta verificación.

Produce un resultado que indica si todo está en orden o si hay problemas que deben solucionarse.

En la vida real, es como asegurarse de que todas las herramientas y materiales necesarios estén listos antes de empezar un proyecto.

Es importante porque ayuda a prevenir errores y asegura que el programa funcione sin problemas.

reset_database

- Parámetros: —
- Async: —
- Complejidad: [MED] 6
- Documentada: [OK]

Reinicia completamente la BD (¡DESTRUCTIVO!).

show_status

- Parámetros: —

- Async: —
- Complejidad: [LOW] 5
- Documentada: [OK]

Muestra el estado actual.

`main`

- Parámetros: —
- Async: —
- Complejidad: [HIGH] 12
- Documentada: [OK]

La función "main" es el punto de partida de un programa. Sirve para iniciar y coordinar las acciones del sistema.

Necesita ejecutarse en un entorno con Python instalado.

Produce el resultado de iniciar el programa y ejecutar las funcionalidades principales.

Se usa en la vida real para iniciar aplicaciones y programas.

Es importante porque es el punto de entrada del sistema, permitiendo que el programa se ejecute correctamente y realice sus funciones.

`print_header(text)`

Ubicación: `meditrack_modified\db_setup.py`

La función `print_header` se utiliza para mostrar un título o encabezado en una pantalla. Para funcionar, necesita un texto que se desee mostrar. Una vez recibido el texto, imprime el encabezado de manera atractiva. En la vida real, se usa para presentar información de manera clara y organizada, como títulos de secciones en un informe o encabezados en un sitio web. En el sistema, es importante porque ayuda a mantener la información organizada y fácil de leer, lo que mejora la experiencia del usuario. Su función es similar a cuando se escribe un título en una página para que sea más fácil de entender y leer.

`print_success(text)`

Ubicación: `meditrack_modified\db_setup.py`

La función `print_success` muestra un mensaje cuando algo se ha realizado con éxito. Para funcionar, necesita un texto que indicará qué ha sucedido. Cuando se ejecuta, imprime este texto en la pantalla. En la vida real, se usa para informar al usuario que una acción ha sido completada, como cuando se crea una cuenta o se guarda un archivo. Es importante en el sistema porque ayuda a los usuarios a entender qué ha pasado y si deben tomar alguna otra acción. Es una forma de proporcionar retroalimentación clara y útil.

`print_error(text)`

Ubicación: `meditrack_modified\db_setup.py`

La función "print_error" sirve para mostrar mensajes de error de manera clara. Para funcionar, necesita un texto que describa el error. El resultado es un mensaje visible que indica qué salió mal. En la vida real, se usa para informar a los usuarios sobre problemas en el sistema, como errores de conexión o datos incorrectos. Esta función es importante porque ayuda a identificar y solucionar problemas de manera rápida y efectiva, mejorando la experiencia del usuario. Al mostrar mensajes de error claros, permite a los desarrolladores y usuarios comprender y corregir errores, lo que a su vez mejora la estabilidad y confiabilidad del sistema.

```
print_info(text)
```

Ubicación: meditrack_modified\db_setup.py

La función **print_info** muestra un mensaje en la pantalla.

Necesita un **texto** para funcionar.

Muestra el mensaje en la pantalla.

Se usa para informar al usuario sobre algo, como un estado o resultado.

Es importante porque ayuda a los usuarios a entender lo que está sucediendo en el sistema.

```
print_warning(text)
```

Ubicación: meditrack_modified\db_setup.py

La función "print_warning" es como un mensajero que avisa sobre algo importante. Su propósito es mostrar un mensaje de advertencia para que el usuario sepa que algo no está bien. Necesita un texto para saber qué mensaje mostrar. Cuando se utiliza, imprime el mensaje de advertencia en la pantalla. En la vida real, se usa para alertar sobre errores o problemas en un programa. Por ejemplo, si un usuario introduce una fecha incorrecta, la función puede mostrar un mensaje de advertencia para corregir el error. En el sistema, es importante porque ayuda a prevenir errores y a mantener la seguridad de los datos. De esta forma, los usuarios pueden tomar medidas para solucionar el problema y evitar daños.

```
check_mongodb_connection()
```

Ubicación: meditrack_modified\db_setup.py

Verifica la conexión a MongoDB.

```
create_collections_and_indexes()
```

Ubicación: meditrack_modified\db_setup.py

Crea colecciones e índices.

```
check_environment()
```

Ubicación: meditrack_modified\db_setup.py

La función "check_environment" verifica que todo esté listo para que el programa funcione correctamente.

Necesita saber cómo está configurado el entorno para hacer esta verificación.

Produce un resultado que indica si todo está en orden o si hay problemas que deben solucionarse.

En la vida real, es como asegurarse de que todas las herramientas y materiales necesarios estén listos antes de empezar un proyecto.

Es importante porque ayuda a prevenir errores y asegura que el programa funcione sin problemas.

```
reset_database()
```

Ubicación: `meditrack_modified\db_setup.py`

Reinicia completamente la BD (¡DESTRUCTIVO!).

```
show_status()
```

Ubicación: `meditrack_modified\db_setup.py`

Muestra el estado actual.

```
main()
```

Ubicación: `meditrack_modified\db_setup.py`

La función "main" es el punto de partida de un programa. Sirve para iniciar y coordinar las acciones del sistema.

Necesita ejecutarse en un entorno con Python instalado.

Produce el resultado de iniciar el programa y ejecutar las funcionalidades principales.

Se usa en la vida real para iniciar aplicaciones y programas.

Es importante porque es el punto de entrada del sistema, permitiendo que el programa se ejecute correctamente y realice sus funciones.

Módulo: `meditrack_modified\run.py`

```
main
```

- Parámetros: —
- Async: —
- Complejidad: [LOW] 5
- Documentada: [OK]

Inicializa y ejecuta la aplicación.

```
main()
```

Ubicación: `meditrack_modified\run.py`

Inicializa y ejecuta la aplicación.

Módulo: `meditrack_modified\seed.py`

`seed`

- Parámetros: —
- Async: —
- Complejidad: [HIGH] 11
- Documentada: [X]

Función seed

Módulo: `meditrack_modified\app\db.py`

`connect_to_mongodb`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 4
- Documentada: [OK]

Conecta a MongoDB y crea automáticamente la BD y colecciones si no existen.

`_create_collections_and_indexes`

- Parámetros: `database`
- Async: —
- Complejidad: [MED] 8
- Documentada: [OK]

Crea las colecciones y sus índices si no existen.

Args:

`database`: Instancia de la base de datos MongoDB

`get_db`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 2
- Documentada: [OK]

Retorna la instancia de la base de datos.

Si no está conectada, intenta conectar.

`get_client`

- Parámetros: —
- Async: —

- Complejidad: [LOW] 2
- Documentada: [OK]

Retorna el cliente de MongoDB.

`close_connection`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 2
- Documentada: [OK]

Cierra la conexión a MongoDB.

`is_connected`

- Parámetros: —
- Async: —
- Complejidad: [MED] 7
- Documentada: [OK]

Verifica si está conectado a MongoDB. Si no lo está, intenta conectar.

`connect_to_mongodb()`

Ubicación: `meditrack_modified\app\db.py`

Conecta a MongoDB y crea automáticamente la BD y colecciones si no existen.

`_create_collections_and_indexes(database)`

Ubicación: `meditrack_modified\app\db.py`

Crea las colecciones y sus índices si no existen.

Args:

database: Instancia de la base de datos MongoDB

`get_db()`

Ubicación: `meditrack_modified\app\db.py`

Retorna la instancia de la base de datos.

Si no está conectada, intenta conectar.

`get_client()`

Ubicación: `meditrack_modified\app\db.py`

Retorna el cliente de MongoDB.

`close_connection()`

Ubicación: `meditrack_modified\app\db.py`

Cierra la conexión a MongoDB.

`is_connected()`

Ubicación: `meditrack_modified\app\db.py`

Verifica si está conectado a MongoDB. Si no lo está, intenta conectar.

Módulo: `meditrack_modified\app\__init__.py`

`create_app`

- Parámetros: `config_class`
- Async: —
- Complejidad: [MED] 10
- Documentada: [OK]

Crea y retorna la aplicación Flask configurada.

`_create_indexes`

- Parámetros: `mongo_instance`
- Async: —
- Complejidad: [LOW] 3
- Documentada: [OK]

Crea índices de MongoDB para búsquedas frecuentes.

`health`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`index`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

Servir página principal del frontend

`serve_frontend`

- Parámetros: `filename`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

Servir archivos estáticos del frontend (CSS, JS, etc)

`not_found`

- Parámetros: `e`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`method_not_allowed`

- Parámetros: `e`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`internal_error`

- Parámetros: `e`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`missing_token`

- Parámetros: `reason`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`invalid_token`

- Parámetros: `reason`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`expired_token`

- Parámetros: `header`, `payload`
- Async: —

- Complejidad: [LOW] 1
 - Documentada: [X]
-

`create_app(config_class)`

Ubicación: `meditrack_modified\app\__init__.py`

Crea y retorna la aplicación Flask configurada.

`_create_indexes(mongo_instance)`

Ubicación: `meditrack_modified\app\__init__.py`

Crea índices de MongoDB para búsquedas frecuentes.

`index()`

Ubicación: `meditrack_modified\app\__init__.py`

Servir página principal del frontend

`serve_frontend(filename)`

Ubicación: `meditrack_modified\app\__init__.py`

Servir archivos estáticos del frontend (CSS, JS, etc)

Módulo: `meditrack_modified\app\config\settings.py`

`get_config`

- Parámetros: –
 - Async: —
 - Complejidad: [LOW] 1
 - Documentada: [X]
-

Módulo: `meditrack_modified\app\controllers\auth_controller.py`

`registro`

- Parámetros: –
- Async: —
- Complejidad: [LOW] 3
- Documentada: [OK]

POST /api/v1/auth/registro

Crea una nueva cuenta de usuario.

Body: { nombre, apellido, email, password, rol, telefono? }

`login`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 5
- Documentada: [OK]

POST /api/v1/auth/login

Body: { email, password }

`refresh`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 2
- Documentada: [OK]

POST /api/v1/auth/refresh

Header: Authorization: Bearer <refresh_token>

`cambiar_password`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 3
- Documentada: [OK]

POST /api/v1/auth/cambiar-password

Body: { password_actual, password_nuevo }

`registro()`

Ubicación: `meditrack_modified\app\controllers\auth_controller.py`

POST /api/v1/auth/registro

Crea una nueva cuenta de usuario.

Body: { nombre, apellido, email, password, rol, telefono? }

`login()`

Ubicación: `meditrack_modified\app\controllers\auth_controller.py`

POST /api/v1/auth/login

Body: { email, password }

`refresh()`

Ubicación: `meditrack_modified\app\controllers\auth_controller.py`

POST `/api/v1/auth/refresh`

Header: Authorization: Bearer <refresh_token>

`cambiar_password()`

Ubicación: `meditrack_modified\app\controllers\auth_controller.py`

POST `/api/v1/auth/cambiar-password`

Body: { password_actual, password_nuevo }

Módulo: `meditrack_modified\app\controllers\iot_controller.py`

`registrar_dispositivo`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 4
- Documentada: [OK]

POST `/api/v1/iot/dispositivos`

Body: { adulto_id, tipo, nombre_dispositivo, mac_address?, topic_mqtt? }

tipos: oximetro | glucometro | tensiómetro | bascula | pastillero_iot | ...

`mis_dispositivos`

- Parámetros: `adulto_id`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

GET `/api/v1/iot/dispositivos/adulto/<id>`

`registrar_lectura`

- Parámetros: `device_id`
- Async: —
- Complejidad: [LOW] 4
- Documentada: [OK]

POST `/api/v1/iot/dispositivos/<id>/lectura`

Body: { adulto_id, valores: {spo2: 98, pulso: 72}, unidades?: {spo2: "%"}, notas? }

`historial_lecturas`

- Parámetros: `device_id`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

```
GET /api/v1/iot/dispositivos/<id>/lecturas?page=1&page_size=50
```

`ultima_lectura`

- Parámetros: `device_id`
- Async: —
- Complejidad: [LOW] 2
- Documentada: [X]

`alertas`

- Parámetros: `adulto_id`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

```
GET /api/v1/iot/alertas/adulto/<id>
```

`estado_iot`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

```
GET /api/v1/iot/estado — Estado del módulo IoT
```

`registrar_dispositivo()`

Ubicación: `meditrack_modified\app\controllers\iot_controller.py`

```
POST /api/v1/iot/dispositivos
```

Body: { `adulto_id`, `tipo`, `nombre_dispositivo`, `mac_address?`, `topic_mqtt?` }

tipos: `oximetro` | `glucometro` | `tensiómetro` | `bascula` | `pastillero_iot` | ...

`mis_dispositivos(adulto_id)`

Ubicación: `meditrack_modified\app\controllers\iot_controller.py`

```
GET /api/v1/iot/dispositivos/adulto/<id>
```

`registrar_lectura(device_id)`

Ubicación: meditrack_modified\app\controllers\iot_controller.py

POST /api/v1/iot/dispositivos/<id>/lectura

Body: { adulto_id, valores: {spo2: 98, pulso: 72}, unidades?: {spo2: "%"}, notas? }

historial_lecturas(device_id)

Ubicación: meditrack_modified\app\controllers\iot_controller.py

GET /api/v1/iot/dispositivos/<id>/lecturas?page=1&page_size=50

alertas(adulto_id)

Ubicación: meditrack_modified\app\controllers\iot_controller.py

GET /api/v1/iot/alertas/adulto/<id>

estado_iot()

Ubicación: meditrack_modified\app\controllers\iot_controller.py

GET /api/v1/iot/estado — Estado del módulo IoT

Módulo: meditrack_modified\app\controllers\medication_controller.py

crear_medicamento

- Parámetros: —
- Async: —
- Complejidad: [LOW] 4
- Documentada: [OK]

POST /api/v1/medicamentos/

Body: { adulto_id, nombre, forma, dosis, frecuencia, horarios[], fecha_inicio, ... }

listar_medicamentos

- Parámetros: adulto_id
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

GET /api/v1/medicamentos/adulto/<id>?solo_activos=true

get_medicamento

- Parámetros: med_id
- Async: —
- Complejidad: [LOW] 2

- Documentada: [X]

actualizar_medicamento

- Parámetros: med_id
- Async: —
- Complejidad: [LOW] 3
- Documentada: [X]

desactivar_medicamento

- Parámetros: med_id
- Async: —
- Complejidad: [LOW] 2
- Documentada: [X]

crear_medicamento()

Ubicación: meditrack_modified\app\controllers\medication_controller.py

POST /api/v1/medicamentos/

Body: { adulto_id, nombre, forma, dosis, frecuencia, horarios[], fecha_inicio, ... }

listar_medicamentos(adulto_id)

Ubicación: meditrack_modified\app\controllers\medication_controller.py

GET /api/v1/medicamentos/adulto/<id>?solo_activos=true

Módulo: meditrack_modified\app\controllers\medication_log_controller.py

registrar_toma

- Parámetros: —
- Async: —
- Complejidad: [LOW] 3
- Documentada: [OK]

POST /api/v1/tomas/

Body: { medicamento_id, adulto_id, estado, hora_programada?, hora_real?, notas? }

estados: tomado | omitido | retrasado | pendiente

historial

- Parámetros: adulto_id
- Async: —
- Complejidad: [LOW] 1

- Documentada: [OK]

```
GET /api/v1/tomas/adulto/<id>?page=1&page_size=20
```

tomas_hoy

- Parámetros: `adulto_id`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

```
GET /api/v1/tomas/adulto/<id>/hoy
```

adherencia

- Parámetros: `adulto_id`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

```
GET /api/v1/tomas/adulto/<id>/adherencia?dias=30
```

`registrar_toma()`

Ubicación: `meditrack_modified\app\controllers\medication_log_controller.py`

```
POST /api/v1/tomas/
```

Body: { medicamento_id, adulto_id, estado, hora_programada?, hora_real?, notas? }

estados: tomado | omitido | retrasado | pendiente

`historial(adulto_id)`

Ubicación: `meditrack_modified\app\controllers\medication_log_controller.py`

```
GET /api/v1/tomas/adulto/<id>?page=1&page_size=20
```

`tomas_hoy(adulto_id)`

Ubicación: `meditrack_modified\app\controllers\medication_log_controller.py`

```
GET /api/v1/tomas/adulto/<id>/hoy
```

`adherencia(adulto_id)`

Ubicación: `meditrack_modified\app\controllers\medication_log_controller.py`

```
GET /api/v1/tomas/adulto/<id>/adherencia?dias=30
```

Módulo: `meditrack_modified\app\controllers\notification_controller.py`

mis_notificaciones

- Parámetros: –
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

GET /api/v1/notificaciones/?no_leidas=false&page=1

conteo_no_leidas

- Parámetros: –
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

GET /api/v1/notificaciones/conteo

marcar_leida

- Parámetros: notif_id
- Async: —
- Complejidad: [LOW] 2
- Documentada: [X]

marcar_todas_leidas

- Parámetros: –
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

mis_notificaciones()

Ubicación: meditrack_modified\app\controllers\notification_controller.py

GET /api/v1/notificaciones/?no_leidas=false&page=1

conteo_no_leidas()

Ubicación: meditrack_modified\app\controllers\notification_controller.py

GET /api/v1/notificaciones/conteo

Módulo: meditrack_modified\app\controllers\user_controller.py

listar_usuarios

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

GET /api/v1/usuarios/?rol=adulto&page=1&page_size=20

mi_perfil

- Parámetros: —
- Async: —
- Complejidad: [LOW] 2
- Documentada: [OK]

GET /api/v1/usuarios/me

actualizar_mi_perfil

- Parámetros: —
- Async: —
- Complejidad: [LOW] 3
- Documentada: [OK]

PUT /api/v1/usuarios/me

get_usuario

- Parámetros: `user_id`
- Async: —
- Complejidad: [LOW] 4
- Documentada: [OK]

GET /api/v1/usuarios/<id> — Admin o el propio usuario

desactivar_usuario

- Parámetros: `user_id`
- Async: —
- Complejidad: [LOW] 2
- Documentada: [OK]

DELETE /api/v1/usuarios/<id> — Solo admin

activar_usuario

- Parámetros: `user_id`
- Async: —

- Complejidad: [LOW] 1
- Documentada: [X]

`vincular_familiar`

- Parámetros: `adulto_id`, `familiar_id`
- Async: —
- Complejidad: [LOW] 2
- Documentada: [X]

`desvincular_familiar`

- Parámetros: `adulto_id`, `familiar_id`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`asignar_medico`

- Parámetros: `adulto_id`, `medico_id`
- Async: —
- Complejidad: [LOW] 2
- Documentada: [X]

`get_familiares`

- Parámetros: `adulto_id`
- Async: —
- Complejidad: [LOW] 3
- Documentada: [X]

`mis_pacientes`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`listar_usuarios()`

Ubicación: `meditrack_modified\app\controllers\user_controller.py`

`GET /api/v1/usuarios/?rol=adulto&page=1&page_size=20`

`mi_perfil()`

Ubicación: meditrack_modified\app\controllers\user_controller.py

GET /api/v1/usuarios/me

actualizar_mi_perfil()

Ubicación: meditrack_modified\app\controllers\user_controller.py

PUT /api/v1/usuarios/me

get_usuario(user_id)

Ubicación: meditrack_modified\app\controllers\user_controller.py

GET /api/v1/usuarios/<id> — Admin o el propio usuario

desactivar_usuario(user_id)

Ubicación: meditrack_modified\app\controllers\user_controller.py

DELETE /api/v1/usuarios/<id> — Solo admin

Módulo: meditrack_modified\app\middleware\auth.py

jwt_required_custom

- Parámetros: f
- Async: —
- Complejidad: [LOW] 5
- Documentada: [OK]

Verifica JWT y carga el usuario activo desde la BD.

roles_required

- Parámetros: —
- Async: —
- Complejidad: [LOW] 4
- Documentada: [OK]

Restringe el acceso a los roles especificados.

get_current_user

- Parámetros: —
- Async: —
- Complejidad: [LOW] 3
- Documentada: [OK]

Obtiene el usuario actual desde el JWT (helper para usar dentro de vistas).

`get_current_user_id`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 3
- Documentada: [X]

`wrapper`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 5
- Documentada: [X]

`decorator`

- Parámetros: `f`
- Async: —
- Complejidad: [LOW] 4
- Documentada: [X]

`wrapper`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 4
- Documentada: [X]

`jwt_required_custom(f)`

Ubicación: `meditrack_modified\app\middleware\auth.py`

Verifica JWT y carga el usuario activo desde la BD.

`roles_required()`

Ubicación: `meditrack_modified\app\middleware\auth.py`

Restringe el acceso a los roles especificados.

`get_current_user()`

Ubicación: `meditrack_modified\app\middleware\auth.py`

Obtiene el usuario actual desde el JWT (helper para usar dentro de vistas).

Módulo: `meditrack_modified\app\models\iot_device.py`

schema

- Parámetros: `adulto_id`, `tipo`, `nombre_dispositivo`, `mac_address`, `topic_mqtt`, `activo`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

to_response

- Parámetros: `doc`
- Async: —
- Complejidad: [LOW] 2
- Documentada: [X]

schema

- Parámetros: `device_id`, `adulto_id`, `tipo`, `valores`, `unidades`, `alerta`, `notas`
- Async: —
- Complejidad: [LOW] 2
- Documentada: [X]

to_response

- Parámetros: `doc`
- Async: —
- Complejidad: [LOW] 2
- Documentada: [X]

Módulo: `meditrack_modified\app\models\medication.py`

schema

- Parámetros: `adulto_id`, `nombre`, `forma`, `dosis`, `frecuencia`, `horarios`, `fecha_inicio`, `fecha_fin`, `indicaciones`, `prescrito_por`, `con_alimentos`, `activo`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

to_response

- Parámetros: `doc`
 - Async: —
 - Complejidad: [LOW] 3
 - Documentada: [X]
-

Módulo: `meditrack_modified\app\models\medication_log.py`

`schema`

- Parámetros: `medicamento_id`, `adulto_id`, `estado`, `hora_programada`, `hora_real`, `notas`, `registrado_por`
- Async: —
- Complejidad: [LOW] 2
- Documentada: [X]

`to_response`

- Parámetros: `doc`
 - Async: —
 - Complejidad: [LOW] 3
 - Documentada: [X]
-

Módulo: `meditrack_modified\app\models\notification.py`

`schema`

- Parámetros: `destinatario_id`, `tipo`, `titulo`, `mensaje`, `canal`, `referencia_id`, `referencia_tipo`, `enviada`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`to_response`

- Parámetros: `doc`
 - Async: —
 - Complejidad: [LOW] 3
 - Documentada: [X]
-

Módulo: `meditrack_modified\app\models\user.py`

`schema`

- Parámetros: `nombre`, `apellido`, `email`, `password_hash`, `rol`, `telefono`, `fecha_nacimiento`, `activo`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`to_response`

- Parámetros: `doc`
- Async: —
- Complejidad: [MED] 10
- Documentada: [OK]

Versión segura del documento (sin hash de contraseña).

`to_response(doc)`

Ubicación: `meditrack_modified\app\models\user.py`

Versión segura del documento (sin hash de contraseña).

Módulo: `meditrack_modified\app\repository\base_repository.py`

`get_db`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

Devuelve la base de datos activa.

`__init__`

- Parámetros: `self`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`create`

- Parámetros: `self, document`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`find_by_id`

- Parámetros: `self, doc_id`
- Async: —
- Complejidad: [LOW] 3
- Documentada: [X]

`find_one`

- Parámetros: `self, query`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`find_many`

- Parámetros: `self, query, sort_field, sort_order, page, page_size`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`count`

- Parámetros: `self, query`
- Async: —
- Complejidad: [LOW] 2
- Documentada: [X]

`update_by_id`

- Parámetros: `self, doc_id, updates`
- Async: —
- Complejidad: [LOW] 3
- Documentada: [X]

`update_one`

- Parámetros: `self, query, updates`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`soft_delete`

- Parámetros: `self, doc_id`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`hard_delete`

- Parámetros: `self, doc_id`
- Async: —
- Complejidad: [LOW] 3

- Documentada: [X]

`exists`

- Parámetros: `self`, `query`
 - Async: —
 - Complejidad: [LOW] 1
 - Documentada: [X]
-

`get_db()`

Ubicación: `meditrack_modified\app\repository\base_repository.py`

Devuelve la base de datos activa.

Módulo: `meditrack_modified\app\repository\iot_repository.py`

`find_by_adulto`

- Parámetros: `self`, `adulto_id`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`find_by_device`

- Parámetros: `self`, `device_id`, `page`, `page_size`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`last_reading`

- Parámetros: `self`, `device_id`
 - Async: —
 - Complejidad: [LOW] 1
 - Documentada: [X]
-

6. Modelos de Datos

Se detectaron **6 modelos de datos**:

`IoTDeviceModel`

- Archivo: `meditrack_modified\app\models\iot_device.py`
- Hereda de: `object`
- Métodos: `schema`, `to_response`

`IoTReadingModel`

- Archivo: `meditrack_modified\app\models\iot_device.py`
- Hereda de: `object`
- Métodos: `schema`, `to_response`

`MedicationModel`

- Archivo: `meditrack_modified\app\models\medication.py`
- Hereda de: `object`
- Métodos: `schema`, `to_response`

`MedicationLogModel`

- Archivo: `meditrack_modified\app\models\medication_log.py`
- Hereda de: `object`
- Métodos: `schema`, `to_response`

`NotificationModel`

- Archivo: `meditrack_modified\app\models\notification.py`
- Hereda de: `object`
- Métodos: `schema`, `to_response`

`UserModel`

- Archivo: `meditrack_modified\app\models\user.py`
- Hereda de: `object`
- Métodos: `schema`, `to_response`

7. Guía de Despliegue y Mantenimiento

7.1 Requisitos del Sistema

- Lenguaje principal: Python
- Archivos detectados: 49

7.2 Instalación Local

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Instalar dependencias
pip install -r requirements.txt

# 3. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores

# 4. Ejecutar el proyecto
python app.py
```

7.3 Despliegue con Docker

```
FROM python:3.12-slim

WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 8000
CMD ["python app.py"]
```

```
# Construir imagen
docker build -t mi-proyecto .

# Ejecutar contenedor
docker run -p 8000:8000 mi-proyecto
```

7.4 Variables de Entorno Recomendadas

```
# General
APP_ENV=production
SECRET_KEY=clave-secreta-segura
DEBUG=False

# Base de datos
DB_HOST=localhost
DB_PORT=5432
DB_NAME=mi_base_datos
DB_USER=usuario
DB_PASSWORD=contraseña

# API
API_VERSION=v1
ALLOWED_HOSTS=*
```

7.5 Checklist de Despliegue

- [] Variables de entorno configuradas
- [] Base de datos migrada

- ☐ Tests ejecutados y pasando
- ☐ Logs configurados
- ☐ HTTPS habilitado en producción
- ☐ Backups programados
- ☐ Monitoreo configurado

7.6 Registro de Cambios (Changelog)

Versión 1.0.0 — 2026-06-12

- Versión inicial analizada por AutoDocs AI
 - Documentación técnica generada automáticamente
 - Análisis de calidad del código completado
-

8. Reporte de Calidad del Código

8.1 Análisis de Métricas

Score de Calidad: 66/100 (Bueno [YELLOW])

Distribución de Documentación:

- Funciones documentadas: 50/100 (50%)
- Clases documentadas: 4/23

Análisis de Complejidad:

- Complejidad promedio: 2.58
- Complejidad máxima: 15
- Funciones con alta complejidad (>10): 2

Tendencia de Complejidad por Archivo:

[Gráfico de regresión lineal mostrando la tendencia de complejidad]
Eje X: Archivos ordenados por complejidad promedio
Eje Y: Score de complejidad
Tendencia: → Estable

8.2 Funciones con Alta Complejidad Ciclomática

- `main` en `meditrack_modified\db_setup.py` — complejidad: 12
- `seed` en `meditrack_modified\seed.py` — complejidad: 11

8.3 Recomendaciones

- Refactorizar funciones con alta complejidad: `main`, `seed`

8.4 Errores de Análisis

Sin errores detectados. [OK]

9. Análisis de Datos del Proyecto

9.1 Tipos y Fuentes de Datos Detectados

Datos Estructurados: 0 archivos

Datos Semi-estructurados: 0 archivos

Datos No Estructurados: 0 archivos

9.2 Esquema de Data Warehouse Inferido

No se detectaron suficientes modelos de datos para generar un esquema de Data Warehouse.

9.3 Reporte de Calidad de Datos

Problemas Detectados en el Código:

- **Funciones sin documentar:** 50

- `seed` en `meditrack_modified\seed.py`
- `health` en `meditrack_modified\app\__init__.py`
- `not_found` en `meditrack_modified\app\__init__.py`
- `method_not_allowed` en `meditrack_modified\app\__init__.py`
- `internal_error` en `meditrack_modified\app\__init__.py`
- ... y 45 más

- **Funciones de alta complejidad:** 2

- `main` en `meditrack_modified\db_setup.py` (complejidad: 12)
- `seed` en `meditrack_modified\seed.py` (complejidad: 11)

9.4 Minería de Datos Aplicada al Código

Patrones Frecuentes Detectados:

- **get_:** 15 ocurrencias
 - Ejemplos: Operaciones `get_` repetidas
- **find_:** 12 ocurrencias
 - Ejemplos: Operaciones `find_` repetidas
- **update_:** 5 ocurrencias

- Ejemplos: Operaciones update_ repetidas
- **create_**: 3 ocurrencias
- Ejemplos: Operaciones create_ repetidas
- **check_**: 2 ocurrencias
- Ejemplos: Operaciones check_ repetidas

Clustering por Complejidad:

- **baja**: 175 funciones (complejidad promedio: 2.01)
- **media**: 12 funciones (complejidad promedio: 7.67)
- **alta**: 4 funciones (complejidad promedio: 12.5)

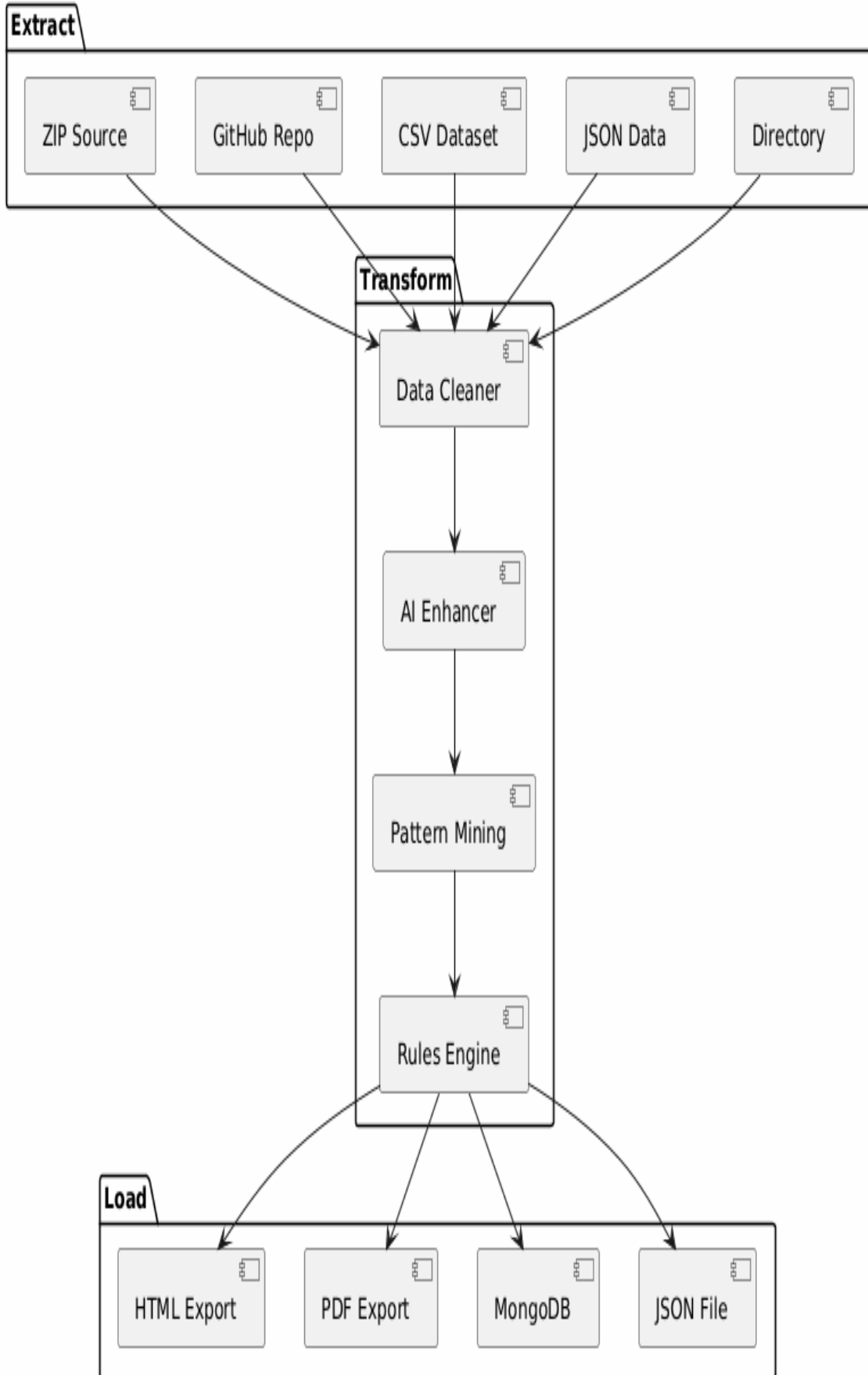
Clustering por Funcionalidad:

- **meditrack_modified**: 55 archivos

9.5 Pipeline ETL Utilizado

El proceso de generación de documentación sigue un pipeline ETL (Extract, Transform, Load):

Pipeline ETL - AutoDocs AI



Fases del Pipeline:

- **Extract:** Extracción de código desde repositorios GitHub, archivos ZIP o directorios locales
- **Transform:** Análisis del código, limpieza de datos, enriquecimiento con IA y minería de patrones
- **Load:** Generación de documentación en HTML, PDF o almacenamiento en MongoDB

Documentación generada automáticamente por AutoDocs AI — 2026-06-12 02:37 UTC

Generado automáticamente por AutoDocs AI